Perplexity

Etiqueta de modelo: (no observada)
Leida: 2026-11-09T21:19:09.141Z
Caracteres: 11996
Fuentes citadas (verificables): 6
Enlaces en el DOM: 6
Fin de respuesta: inferido

Pregunta

Contexto: ChatCouncil es una app de investigación que abre diez páginas web reales de proveedores de IA a la vez, cada una en su propio webview nativo con la sesión del usuario; escribe en sus campos de texto, lee sus respuestas del DOM, adjunta archivos y arma un informe. Se está reescribiendo con el núcleo en Rust y un solo código para Windows, macOS, Linux, Android e iOS (escritorio, móvil y tableta), con el rendimiento (RAM, CPU, tiempo de arranque y tamaño) como prioridad absoluta.

Ya medido en Windows, con diez webviews nativos de WebView2 en la misma máquina: Flutter con plugins que dibujan por textura gasta unos 5 puntos de CPU extra cuando las páginas redibujan; Flutter con ventanas nativas y Slint con ventanas nativas (wry) gastan casi cero en el proceso propio; Slint arranca en 0,76 s contra 3,35 s de Flutter, pesa 11 MiB contra 25,5 MiB y usa 74 MB de RAM en el proceso propio contra 122 MB. Se descartaron Dioxus (su interfaz sería un webview más) y Rune (no tiene Android ni iOS).

Pregunta: ¿qué opciones de interfaz y de manejo de varios webviews nativos simultáneos, viables hoy para esas cinco plataformas con un solo código y un núcleo en Rust, no se están considerando, y cómo se comparan en rendimiento medido con Slint y con Flutter? Para cada opción: fuentes verificables con fecha, estado real en cada plataforma (en especial varios webviews nativos simultáneos en Android e iOS), madurez, licencia y riesgos.

Respuesta

La conclusión operativa: no estás dejando fuera muchas opciones serias, pero sí tres familias relevantes: Makepad, Tauri/wry como shell nativo y UI nativa por plataforma con núcleo Rust compartido. Para tu caso —diez webviews reales, sesión del usuario, automatización DOM y prioridad absoluta de RAM/CPU/arranque— la decisión no es tanto “qué framework Rust dibuja botones” como cómo se alojan los diez webviews: como vistas nativas dentro de una ventana/actividad, o como texturas compuestas por el framework.
Corrección importante

En Android e iOS, “diez webviews nativos simultáneos” normalmente no significa diez ventanas del sistema. Significa diez instancias de android.webkit.WebView o WKWebView embebidas como vistas hijas dentro de una Activity/escena; crear diez ventanas separadas es otra cosa y no resuelve el problema de mostrar diez proveedores a la vez. Tauri v2 añadió multi-ventana móvil mediante Activity Embedding en Android y UIScene en iPadOS, pero eso es para ventanas separadas, no para una grilla de diez webviews en una misma pantalla.v2.tauri+2

Además, “ventanas nativas” en escritorio no equivale a “vistas nativas” en móvil: en móvil el análogo correcto es insertar el WebView nativo en la jerarquía de vistas de la plataforma, no dibujarlo a textura.

Opciones no consideradas

Opción | Estado real en las 5 plataformas | Varios webviews nativos | Madurez | Licencia | Riesgo principal | Tauri v2 + wry | Windows, macOS, Linux, Android, iOS estables desde v2.0 | Escritorio: sí; móvil: multi-webview en una ventana es API inestable; multi-ventana móvil existe desde 2.11 | Alta para desktop, media para móvil | MIT / Apache-2.0 | Tu propia UI sería web; contradice la prioridad de arranque y RAM |

Makepad | Windows, macOS, Linux, Android, iOS con backends propios | No trae abstracción de webview: hay que integrar WebView2/WKWebView/Android WebView por plataforma | Baja-media; activo pero ecosistema pequeño | MIT | DSL propio, documentación dispersa, falta de componentes empresariales |

egui + eframe | Desktop first-class; Android oficial; iOS no es target oficial de eframe | No resuelve webviews; requeriría embedding nativo manual | Alta en desktop, media en Android, baja en iOS | MIT / Apache-2.0 | Repintado inmediato y falta de multi-viewport; mala opción para diez webviews |

Iced | Desktop first-class; móvil mediante proyectos comunitarios, no camino oficial completo | No integración nativa de webviews | Media en desktop, baja en móvil | MIT | Móvil y multi-ventana/webview son trabajo propio |

Xilem | Desktop; Android demo-grade; iOS sin camino | No | Temprana | Apache-2.0 / MIT según componentes | No apto como base de producto multiplataforma hoy |

GPUI | Windows, macOS, Linux; móvil sólo vía gpui-mobile comunitario | No | Baja para uso general multiplataforma | Apache-2.0 con excepciones | Acoplado al diseño de Zed; móvil no es camino soportado |

Freya | Desktop; móvil experimental | No | Baja-media | MIT | Depende de Skia/winit y tiene menor madurez que Slint |

Relm4 / GTK | Linux fuerte; Windows/macOS posibles; Android/iOS no son camino serio | GTK WebView posible en desktop; no móvil | Alta en Linux, baja móvil | MIT | No cubre las cinco plataformas con un solo código |

Núcleo Rust + UI nativa por plataforma | Cada plataforma usa su toolkit idiomático | Sí, con controles nativos reales | Máxima por plataforma | N/A | Rompe “un solo código de UI”; mayor costo de mantenimiento |

CEF embebido | Windows, macOS, Linux; Android/iOS no soportados oficialmente | Sí, pero

Chromium propio | Alta en desktop | BSD y licencias de Chromium | RAM, tamaño y actualizaciones de Chromium; descarta móvil |

Tauri v2 + wry

Es la opción que sí deberías evaluar explícitamente, aunque probablemente no como reemplazo de Slint para la UI propia. Tauri ya resuelve la capa que más duele de tu app —crear y controlar webviews del sistema mediante wry— y en v2 soporta Android e iOS desde la misma base Rust.v2.tauri+2

Multi-webview en una ventana: existe, pero sigue behind the unstable Cargo feature; Tauri documenta que new Webview(...) para varios webviews hijos requiere esa feature y que la API sigue en revisión.v2.tauri+1

Móvil: el soporte de múltiples ventanas llegó en la línea 2.11 mediante Activity Embedding y Scenes; no es equivalente a diez webviews hijos dentro de una sola pantalla.v2.tauri+2

Rendimiento esperado: en desktop, si Tauri dibuja la UI en un webview, pagarías el costo de otro runtime web; el estudio independiente de 2026 midió a Tauri en torno a 14% de un núcleo promedio en su batería, frente a 9,2% de Slint, aunque no es directamente comparable a tu caso de diez webviews.goldstrikearch.github

Veredicto: útil como capa de webview, no como UI principal. Para ChatCouncil, usar Tauri sólo para orquestar webviews y Slint para el shell sería una arquitectura híbrida posible, pero añade dos runtimes de UI y probablemente empeora arranque y RAM frente a Slint + wry directo.

Qué confirmaría la elección: un prototipo con diez Webview hijos en desktop y una prueba de dos a cuatro webviews embebidos en Android/iOS, midiendo RAM, arranque y input latency. Qué lo falsificaría: que los webviews hijos no puedan coexistir con una UI nativa Slint, o que el modo inestable tenga bugs de layout/resize bloqueantes; ya hay issues abiertos de posicionamiento y resize.github

Makepad

Makepad es la alternativa Rust más interesante que no aparece en tu descarte. No usa winit, wgpu ni Taffy: implementa sus propios backends de ventana, renderizador GPU, fuentes y plataforma, incluyendo Windows, macOS, Linux, Android, iOS, WebAssembly y hasta OpenXR.github+1

Estado real: soporta las cinco plataformas mediante backends propios; eso es una ventaja arquitectónica real frente a Slint, que en móvil depende en buena parte de winit/Skia.github

Webviews: no ofrece una capa equivalente a wry lista para producción; tendrías que escribir integración nativa por plataforma: WebView2 en Windows, WKWebView en macOS/iOS, WebKitGTK o similar en Linux y android.webkit.WebView en Android. Eso es viable, pero es ingeniería propia.

Rendimiento esperado: potencialmente igual o mejor que Slint en CPU y RAM, porque evita capas intermedias y renderiza directamente con GPU; pero no hay medición pública comparable a tu benchmark de diez webviews, así que cualquier afirmación de superioridad sería especulación.

Licencia: MIT.github

Riesgo: alto. El proyecto tiene actividad, pero documentación, componentes y ecosistema muy por detrás de Slint; además, tendrías que resolver teclado, IME, accesibilidad, ciclo de vida móvil y webviews manualmente.

Qué confirmaría: un spike de Makepad con una ventana host y dos webviews nativos en Windows y Android. Qué lo falsificaría: que la integración de vistas nativas externas requiera modificar el render loop o que el ciclo de vida Android/iOS sea inestable.

egui / eframe

egui es maduro y excelente para herramientas internas, pero es una mala base para ChatCouncil tal como está definido.

Plataformas: eframe declara Web, Linux, macOS, Windows y Android; iOS no es un target oficial de eframe, aunque puede intentarse con winit y un host propio.github+2

Multi-webview: egui no tiene multi-viewport; integrar diez webviews nativos requeriría una capa propia de composición o ventanas nativas separadas.github

Rendimiento esperado: en el estudio independiente, egui promedió 9,6% de un núcleo, apenas por encima de Slint en 9,2%; pero ese benchmark no mide el costo extra de incrustar webviews por textura, que es precisamente el problema que ya mediste con Flutter.goldstrikearch.github+1

Riesgo: medio-alto. El modo inmediato rehace layout cada frame; para una UI de control con pocos elementos puede ser aceptable, pero no ofrece ventaja frente a Slint y complica el embedding móvil.

Iced, Xilem, GPUI y Freya

Framework | Mobile hoy | Multi-webview | Lectura para ChatCouncil | Iced | No hay camino oficial completo; existe iced_mobile comunitario para iOS/Android | No | No es mejor que Slint para tu requisito; añade riesgo móvil |

Xilem | Android demo-grade; iOS sin soporte | No | Demasiado temprano |

GPUI | Móvil sólo mediante gpui-mobile; no es soporte oficial general | No | Descartable para producto multiplataforma hoy |

Freya | Experimental | No | Interesante técnicamente, pero menos maduro que Slint |

Iced tiene desktop first-class, pero el camino móvil sigue siendo comunitario: iced_mobile existe como prueba de concepto y requiere un runner distinto, no una plataforma soportada de primera clase. Xilem tiene integración Android real pero calificada como demo-grade, y no tiene historia para iOS. GPUI ha mejorado en escritorio, pero su soporte móvil proviene de un proyecto comunitario y no constituye una base estable para las cinco plataformas.github+2

La opción que sí cambia el modelo

Núcleo Rust compartido + UI nativa por plataforma es la única alternativa que elimina por completo el impuesto de renderizado del framework:

Windows: WinUI 3 / Win32 + WebView2.

macOS/iOS: SwiftUI/AppKit + WKWebView.

Linux: GTK 4 + WebKitGTK.

Android: Jetpack Compose o Views + android.webkit.WebView.

El núcleo —automatización, sesiones, extracción DOM, orquestación, informes— permanece en Rust mediante FFI/uniffi.

Esto da el mejor control sobre los diez webviews nativos y probablemente el mejor perfil de RAM/CPU/arranque, porque cada plataforma usa su stack nativo. Pero viola tu restricción de un solo

código de UI y multiplica el costo de mantenimiento; por eso no lo plantearía como reemplazo de Slint, sino como plan B si Slint + wry no logra embeber webviews nativos sin textura en móvil.

Cómo se comparan con tus mediciones

Tus números de Windows son consistentes con la causa técnica: Flutter con textura introduce una etapa extra de composición y sincronización entre el WebView y el renderizador de Flutter; Slint y Flutter con ventanas nativas dejan el webview compuesto por el sistema.goldstrikearch.github

Arquitectura | Costo esperado en proceso propio | Arranque | Riesgo móvil | Adecuación ChatCouncil |
Flutter + webviews por textura | Alto; ya mediste ~5 puntos CPU extra | Alto; 3,35 s medido | Medio | Descartada por rendimiento |
Flutter + ventanas nativas | Muy bajo en proceso propio | Alto; 3,35 s medido | Medio | Descartada por arranque/tamaño |
Slint + wry nativo | Muy bajo; 74 MB RAM y 0,76 s medidos | Bajo | Medio: embedding móvil por confirmar | Mejor opción actual |
Tauri + wry, UI web | Medio-alto por runtime web adicional | Medio | Medio; multi-webview inestable | Sólo como capa de webview |
Makepad + webviews nativos | Potencialmente muy bajo, sin medición | Potencialmente bajo | Alto | Apuesta de alto riesgo |
egui/eframe | Similar o algo peor que Slint en benchmark público | Desconocido para tu app | Alto en iOS | No recomendada |
UI nativa por plataforma | Mínimo | Mínimo | Bajo técnico, alto de mantenimiento | Plan B si Slint falla |

Recomendación práctica

Mantén Slint + wry como apuesta principal, pero convierte “Slint con ventanas nativas” en una prueba explícita de “Slint como shell + diez webviews nativos embebidos como vistas hijas” en Windows, Android y iOS. La incógnita real no es Slint: es si wry puede entregarte diez webviews nativos en una sola superficie móvil sin recurrir a texturas ni a ventanas separadas.

Si eso falla, el orden de contingencia sería:

Tauri/wry sólo como capa de webview, conservando Slint para el shell, si la API inestable de child webviews resulta suficientemente sólida.

Makepad, si aceptas asumir integración nativa y riesgos de madurez a cambio de un stack Rust más ligero.

UI nativa por plataforma con núcleo Rust compartido, si el requisito de rendimiento y de diez webviews simultáneos es innegociable.

La evidencia disponible respalda a Slint como favorito, pero no valida todavía que pueda alojar diez webviews nativos simultáneos en Android e iOS; ese es el experimento que debe decidir la arquitectura, no una comparación genérica de frameworks.v2.tauri+3

Texto capturado tal cual de la interfaz del proveedor, sin corregir ni recortar.

Este documento es la respuesta a la pregunta, no la evaluacion que este

proveedor hizo de las respuestas de los demas.